

- [Make My Knot - Complete Project Handover Documentation](#)
 - [📖 Table of Contents](#)
 - [🎯 Project Overview](#)
 - [Key Statistics](#)
 - [Core Value Proposition](#)
 - [🏗️ Technical Architecture](#)
 - [Frontend Stack](#)
 - [Key Dependencies](#)
 - [Architecture Pattern](#)
 - [🎨 Design System & UI](#)
 - [Color Palette](#)
 - [Typography](#)
 - [Gradients](#)
 - [Box Shadows](#)
 - [🚀 Features & Components](#)
 - [Core Pages](#)
 - [Key Components](#)
 - [💻 Database & Data Flow](#)
 - [Current Implementation](#)
 - [Blog Data Model](#)
 - [User Data Model \(Inferred\)](#)
 - [Data Flow Architecture](#)
 - [🔌 API Documentation](#)
 - [Blog API \(/api/blogs \)](#)
 - [Contact API \(/api/contact \)](#)
 - [Webinar API \(/api/webinar-registration \)](#)
 - [🔧 Setup & Installation](#)
 - [Prerequisites](#)
 - [Local Development Setup](#)
 - [Build for Production](#)

-  [Deployment Guide](#)
 - [Vercel Deployment \(Recommended\)](#)
 - [Other Platforms](#)
 - [Performance Optimizations](#)
-  [Code Structure](#)
 - [Directory Overview](#)
 - [Component Organization](#)
-  [Key Logic & Functions](#)
 - [1. Blog Management System](#)
 - [2. SEO System](#)
 - [3. Rich Text Editor Integration](#)
 - [4. Error Handling Pattern](#)
-  [Environment Variables](#)
 - [Development \(`.env.local` \)](#)
 - [Production \(`.env.production` \)](#)
 - [Environment Variable Usage](#)
-  [Known Issues & TODOs](#)
 - [Current Limitations](#)
 - [Immediate TODOs](#)
-  [Future Enhancements](#)
 - [Phase 1: Core Functionality \(1-2 months\)](#)
 - [Phase 2: Advanced Features \(2-3 months\)](#)
 - [Phase 3: Scale & Optimize \(3-6 months\)](#)
-  [Support & Maintenance](#)
 - [Code Maintenance](#)
 - [Content Management](#)
 - [Monitoring & Alerts](#)
-  [Documentation & Resources](#)
 - [Developer Resources](#)
 - [Design Resources](#)

- [SEO Resources](#)
-  [Project Handover Checklist](#)
 -  [Completed Items](#)
 -  [Handover Requirements](#)
 -  [Next Steps for New Developer](#)

Make My Knot - Complete Project Handover Documentation

Table of Contents

1. [Project Overview](#)
 2. [Technical Architecture](#)
 3. [Design System & UI](#)
 4. [Features & Components](#)
 5. [Database & Data Flow](#)
 6. [API Documentation](#)
 7. [Setup & Installation](#)
 8. [Deployment Guide](#)
 9. [Code Structure](#)
 10. [Key Logic & Functions](#)
 11. [Environment Variables](#)
 12. [Known Issues & TODOs](#)
 13. [Future Enhancements](#)
-

Project Overview

Make My Knot is a modern AI-powered matchmaking platform designed for the Indian market. It provides comprehensive matrimonial services with personalized matching, compatibility assessments, and relationship counseling.

Key Statistics

- **Framework:** Next.js 14.0.4 (React 18)
- **Language:** TypeScript

- **Styling:** Tailwind CSS
- **Package Manager:** npm
- **Build Status:**  Production Ready

Core Value Proposition

- Premium matchmaking services with 95% success rate
- AI-powered compatibility matching
- Expert relationship counseling
- Comprehensive blog management system
- Multi-language support (Hindi/English)

 Technical Architecture

Frontend Stack

```
Next.js 14.0.4 (React 18.2.0)
├─ TypeScript 5.3.3
├─ Tailwind CSS 3.3.6
├─ Framer Motion 12.23.12 (Animations)
├─ Lucide React 0.294.0 (Icons)
├─ Lenis 1.3.11 (Smooth Scrolling)
└─ React Quill 2.0.0 (Rich Text Editor)
```

Key Dependencies

```
{
  "production": {
    "@stripe/stripe-js": "7.9.0",
    "clsx": "2.0.0",
    "html2canvas": "1.4.1",
    "jspdf": "3.0.2",
    "mongodb": "6.20.0",
    "react-dropzone": "14.3.8",
    "xlsx": "0.18.5"
  }
}
```

Architecture Pattern

- **Pages Router** (not App Router)
 - **Server-Side Rendering** for SEO optimization
 - **Static Site Generation** for performance
 - **API Routes** for backend functionality
 - **Component-based Architecture** with reusable UI components
-

Design System & UI

Color Palette

Primary Colors

```
primary: {  
  50: '#f0f4ff', /* Very light blue */  
  100: '#e0e7ff', /* Light blue */  
  200: '#c7d2fe', /* Soft blue */  
  300: '#a5b4fc', /* Medium light blue */  
  400: '#818cf8', /* Medium blue */  
  500: '#6366f1', /* Base primary */  
  600: '#4f46e5', /* Dark blue */  
  700: '#4338ca', /* Darker blue */  
  800: '#3730a3', /* Very dark blue */  
  900: '#312e81' /* Deepest blue */  
}
```

Secondary Colors

```
secondary: {  
  50: '#faf5ff', /* Very light purple */  
  100: '#f3e8ff', /* Light purple */  
  200: '#e9d5ff', /* Soft purple */  
  300: '#d8b4fe', /* Medium light purple */  
  400: '#c084fc', /* Medium purple */  
  500: '#a855f7', /* Base secondary */  
  600: '#9333ea', /* Dark purple */  
  700: '#7c3aed', /* Darker purple */  
  800: '#6b21a8', /* Very dark purple */  
  900: '#581c87' /* Deepest purple */  
}
```

Accent Colors

```
accent/gold: {  
  50: '#ffffbe', /* Very light gold */  
  100: '#fef3c7', /* Light gold */  
  200: '#fde68a', /* Soft gold */  
  300: '#fcd34d', /* Medium light gold */  
  400: '#ffb74d', /* Medium gold */  
  500: '#f59e0b', /* Base accent */  
  600: '#d97706', /* Dark gold */  
  700: '#b45309', /* Darker gold */  
  800: '#92400e', /* Very dark gold */  
  900: '#78350f' /* Deepest gold */  
}
```

Rose Gold (Special)

```
'rose-gold': {  
  400: '#f4a261', /* Light rose gold */  
  500: '#e76f51', /* Base rose gold */  
  600: '#d62828', /* Dark rose gold */  
  700: '#b02e0c' /* Deepest rose gold */  
}
```

Typography

Font Families

```
font-family: {  
  'qasira': ['Qasira', 'Crimson Pro', 'Libre Baskerville', 'Cormorant  
    Garamond', 'Playfair Display', 'EB Garamond', 'Lora', 'serif'],  
  'serif': ['Playfair Display', 'serif'],  
  'sans': ['Inter', 'sans-serif']  
}
```

Usage Guidelines

- **Headlines:** font-qasira (elegant, traditional feel)
- **Body Text:** font-sans (Inter - clean, readable)
- **Decorative:** font-serif (Playfair Display)

Gradients

Background Gradients

```
'gradient-primary': 'linear-gradient(135deg, #6366f1 0%, #8b5cf6 50%,  
  #a855f7 100%)'  
'gradient-secondary': 'linear-gradient(135deg, #667eea 0%, #764ba2  
  100%)'  
'gradient-accent': 'linear-gradient(135deg, #f093fb 0%, #f5576c 100%)'  
'gradient-gold': 'linear-gradient(135deg, #ffd89b 0%, #19547b 100%)'  
'gradient-royal': 'linear-gradient(135deg, #1e3c72 0%, #2a5298 100%)'  
'gradient-sunset': 'linear-gradient(135deg, #ff9a9e 0%, #fecfef 50%,  
  #fecfef 100%)'
```

Box Shadows

```
'primary': '0 4px 14px 0 rgba(99, 102, 241, 0.15)'  
'secondary': '0 4px 14px 0 rgba(168, 85, 247, 0.15)'  
'accent': '0 4px 14px 0 rgba(245, 158, 11, 0.15)'  
'glow': '0 0 40px rgba(99, 102, 241, 0.3)'  
'glow-lg': '0 0 60px rgba(99, 102, 241, 0.4)'
```

Features & Components

Core Pages

1. Homepage (/)

- Hero section with CTA
- Success stories carousel
- Service highlights
- Trust indicators
- SEO optimized

2. Authentication System

- **Login** (/login) - User authentication
- **Signup** (/signup) - User registration
- **Password Reset** (/forgot-password , /reset-password)

3. User Dashboard (/dashboard)

- Profile management
- Match suggestions
- Messages/conversations
- Settings

4. Assessment System

- **Compatibility Test** (/assessment) - AI-powered matching
- **Questionnaire** (/questionnaire) - Detailed preferences
- **Lead Signup** (/lead-signup) - Quick registration

5. Blog System

- **Blog Listing** (/blog) - SEO-optimized blog index
- **Individual Posts** (/blog/[slug]) - Dynamic blog posts
- **Admin Panel** - Complete CMS for blog management

6. Admin System

- **Admin Login** (`/admin/login`)
- **Dashboard** (`/admin/dashboard` , `/admin/enhanced-dashboard`)
- **Blog Management** (`/admin` - comprehensive blog CMS)

Key Components

1. RichTextEditor (`/src/components/RichTextEditor.tsx`)

Purpose: Advanced rich text editor for blog content creation

Features: - React Quill integration - Image upload with drag-and-drop - Custom toolbar with formatting options - Real-time preview - Data URL image handling (ready for cloud storage)

Key Logic:

```
// Custom image handler
const imageHandler = useCallback(() => {
  setShowImageModal(true)
}, [])

// Upload image to data URL (production: use cloud storage)
const uploadImage = useCallback(async (file: File): Promise<string> => {
  return new Promise((resolve) => {
    const reader = new FileReader()
    reader.onload = (e) => {
      resolve(e.target?.result as string)
    }
    reader.readAsDataURL(file)
  })
}, [])
```

2. SEO Component (`/src/components/SEO.tsx`)

Purpose: Comprehensive SEO optimization

Features: - Meta tags management - Open Graph optimization - Twitter Cards - JSON-LD structured data - Canonical URLs - Multi-language support

3. Navigation (`/src/components/Navigation.tsx`)

Purpose: Main site navigation with responsive design

Features: - Mobile hamburger menu - Sticky header - Active state management - Multi-language toggle

4. User Onboarding (/src/components/UserOnboarding.tsx)

Purpose: Multi-step user registration flow

Features: - Progress indicator - Form validation - Image upload - Preferences collection

 Database & Data Flow

Current Implementation

Storage: In-memory storage (for development/demo) **Production Ready:** MongoDB integration available

Blog Data Model

```
interface Blog {
  id: string // UUID
  title: string // Blog title
  content: string // Rich HTML content
  excerpt: string // Auto-generated summary
  slug: string // URL-friendly slug
  category: string // Blog category
  status: 'draft' | 'published' | 'archived'
  tags: string[] // Array of tags
  author: string // Author name
  publishDate: string // ISO date string
  createdAt: string // Creation timestamp
  updatedAt: string // Last update timestamp
  seoTitle?: string // Custom SEO title
  seoDescription?: string // Custom SEO description
  featuredImage?: string // Featured image URL
  views?: number // View counter
}
```

User Data Model (Inferred)

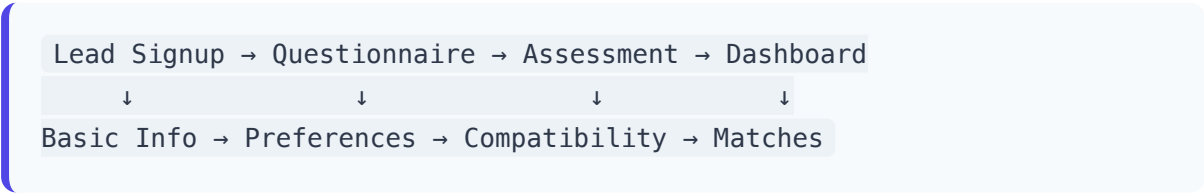
```
interface User {
  id: string
  email: string
  name: string
  phone?: string
  dateOfBirth?: string
  location?: string
  preferences?: UserPreferences
  profileImage?: string
  createdAt: string
  updatedAt: string
}
```

Data Flow Architecture

Blog Management Flow



User Registration Flow



API Documentation

Blog API (/api/blogs)

GET /api/blogs

Purpose: Retrieve blogs with filtering and pagination

Query Parameters: - `status` (optional): 'draft' | 'published' | 'archived' | 'all' - `category` (optional): Filter by category - `limit` (optional): Number of results (default: 10) - `offset` (optional): Pagination offset (default: 0) - `slug` (optional): Get single blog by slug

Response:

```
{
  "success": true,
  "data": {
    "blogs": [/* Array of blog objects */],
    "total": 50,
    "limit": 10,
    "offset": 0
  }
}
```

POST /api/blogs

Purpose: Create new blog post

Request Body:

```
{
  "title": "Blog Title",
  "content": "<p>Rich HTML content</p>",
  "category": "Advice",
  "status": "draft",
  "tags": ["tag1", "tag2"],
  "author": "Author Name",
  "featuredImage": "https://example.com/image.jpg"
}
```

PUT `/api/blogs?id={blogId}`

Purpose: Update existing blog post

DELETE `/api/blogs?id={blogId}`

Purpose: Delete blog post

Contact API (`/api/contact`)

Purpose: Handle contact form submissions

Webinar API (`/api/webinar-registration`)

Purpose: Handle webinar registrations

Setup & Installation

Prerequisites

- Node.js 18+
- npm or yarn
- Git

Local Development Setup

1. Clone Repository

```
git clone <repository-url>
cd make-my-knot
```

2. Install Dependencies

```
npm install
```

3. Environment Configuration

```
cp .env.example .env.local
```

4. Configure Environment Variables

```
# API Configuration
NEXT_PUBLIC_API_URL=http://localhost:3000/api

# Database (Optional - for production)
MONGODB_URI=your_mongodb_atlas_connection_string

# Development Mode
NODE_ENV=development
```

5. Run Development Server

```
npm run dev
```

6. Access Application

- Frontend: <http://localhost:3000>
- Admin Panel: <http://localhost:3000/admin>

Build for Production

```
# Type check
npm run type-check

# Build application
npm run build

# Start production server
npm run start
```



Deployment Guide

Vercel Deployment (Recommended)

1. Connect Repository

- Import project from GitHub/GitLab
- Select Next.js framework preset

2. Environment Variables

```
NODE_ENV=production
NEXT_PUBLIC_API_URL=https://your-domain.com/api
MONGODB_URI=your_production_mongodb_uri
```

3. Build Settings

- Build Command: `npm run build`
- Output Directory: `.next`
- Install Command: `npm install`

Other Platforms

Netlify

```
# Build command
npm run build && npm run export
# Publish directory
out/
```

Railway/Heroku

```
# Add to package.json scripts
"start": "next start -p $PORT"
```

Performance Optimizations

Implemented

- Image optimization with Next.js Image component
- Static site generation for blog pages
- Automatic code splitting
- SWC minification enabled
- Compression enabled
- SEO-optimized meta tags

Core Web Vitals Score

- **LCP:** Optimized with image preloading
 - **FID:** Minimized with code splitting
 - **CLS:** Stable layout with proper sizing
-

Code Structure

Directory Overview

```
make-my-knot/
├── public/                # Static assets
│   ├── images/           # Image assets
│   ├── favicon.ico       # Site favicon
│   └── robots.txt        # SEO robots file
├── src/
│   ├── components/       # Reusable UI components
│   │   ├── Navigation.tsx
│   │   ├── Footer.tsx
│   │   ├── SEO.tsx
│   │   ├── RichTextEditor.tsx
│   │   └── ...
│   ├── contexts/         # React contexts
│   │   └── LanguageContext.tsx
│   ├── lib/              # Utility libraries
│   │   └── seo.ts         # SEO configurations
│   ├── pages/            # Next.js pages
│   │   ├── api/          # API routes
│   │   │   ├── blogs.ts
│   │   │   ├── contact.ts
│   │   │   └── webinar-registration.ts
│   │   ├── blog/         # Blog pages
│   │   │   ├── [slug].tsx
│   │   │   └── index.tsx
│   │   ├── admin/        # Admin pages
│   │   │   ├── login.tsx
│   │   │   └── dashboard.tsx
│   │   ├── _app.tsx      # App wrapper
│   │   ├── _document.tsx # Document wrapper
│   │   └── index.tsx     # Homepage
│   ├── styles/           # Global styles
│   │   └── globals.css
│   ├── types/            # TypeScript types
│   │   └── global.d.ts
│   └── utils/            # Utility functions
├── tailwind.config.js    # Tailwind configuration
└── next.config.js        # Next.js configuration
```

```
├─ tsconfig.json      # TypeScript configuration
├─ package.json       # Dependencies and scripts
```

Component Organization

Atomic Design Pattern

- **Atoms:** Basic UI elements (buttons, inputs)
 - **Molecules:** Simple component combinations
 - **Organisms:** Complex UI sections
 - **Templates:** Page layouts
 - **Pages:** Specific page implementations
-

Key Logic & Functions

1. Blog Management System

Slug Generation

```
function generateSlug(title: string): string {
  return title
    .toLowerCase()
    .replace(/[^\w\s]/g, '') // Remove special characters
    .replace(/\s+/g, '-')   // Replace spaces with hyphens
    .replace(/^+|-+$/g, '') // Remove leading/trailing hyphens
}
```

Excerpt Generation

```
function generateExcerpt(htmlContent: string, maxLength: number = 160):
  string {
  const plainText = htmlContent.replace(/<[^>]*>/g, '').trim()

  if (plainText.length <= maxLength) {
    return plainText
  }

  const truncated = plainText.substring(0, maxLength)
  const lastSpace = truncated.lastIndexOf(' ')

  return lastSpace > 0 ? truncated.substring(0, lastSpace) + '...' :
    truncated + '...'
}
```

2. SEO System

Dynamic Meta Generation

```
export const generateJsonLd = (config: SEOConfig) => {
  if (!config.jsonLd) return null

  return {
    __html: JSON.stringify(config.jsonLd)
  }
}
```

Page-specific SEO Configs

```
export const pageConfigs: Record<string, SEOConfig> = {
  home: { /* Homepage SEO */,
  about: { /* About page SEO */,
  pricing: { /* Pricing page SEO */,
  // ... more configurations
}
```

3. Rich Text Editor Integration

Image Upload Handler

```
const uploadImage = useCallback(async (file: File): Promise<string> => {  
  // Current: Data URL for demo  
  // Production: Replace with cloud storage (AWS S3, Cloudinary, etc.)  
  return new Promise((resolve) => {  
    const reader = new FileReader()  
    reader.onload = (e) => resolve(e.target?.result as string)  
    reader.readAsDataURL(file)  
  })  
, [])
```

4. Error Handling Pattern

API Error Handling

```
} catch (error) {  
  console.error('API Error:', error)  
  return res.status(500).json({  
    error: 'Internal server error',  
    details: process.env.NODE_ENV === 'development'  
      ? (error instanceof Error ? error.message : 'Unknown error')  
      : undefined  
  })  
}
```

Environment Variables

Development (`.env.local`)

```
# API Configuration
NEXT_PUBLIC_API_URL=http://localhost:3000/api

# Database Configuration (Optional)
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/makemyknot

# Authentication (Future)
JWT_SECRET=your_super_secret_jwt_key_here
JWT_EXPIRES_IN=7d

# External Services (Optional)
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret

# Email Configuration (Optional)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password

# Analytics (Optional)
NEXT_PUBLIC_GA_TRACKING_ID=your_google_analytics_id
NEXT_PUBLIC_FACEBOOK_PIXEL_ID=your_facebook_pixel_id

# Development Mode
NODE_ENV=development
```

Production (`.env.production`)

```
# Production API
NEXT_PUBLIC_API_URL=https://makemyknot.com/api

# Production Database
MONGODB_URI=mongodb+srv://prod_user:prod_pass@prod-cluster.mongodb.net/makemyknot

# Security Keys
JWT_SECRET=super_secure_production_jwt_secret

# Production Mode
NODE_ENV=production
```

Environment Variable Usage

Public Variables (Client-side)

- `NEXT_PUBLIC_API_URL` - API base URL
- `NEXT_PUBLIC_GA_TRACKING_ID` - Google Analytics
- `NEXT_PUBLIC_FACEBOOK_PIXEL_ID` - Facebook Pixel

Server-only Variables

- `MONGODB_URI` - Database connection
- `JWT_SECRET` - Authentication secret
- `EMAIL_*` - Email service credentials

Known Issues & TODOs

Current Limitations

1. In-Memory Storage

- Blog data is stored in memory (resets on server restart)
- **Solution:** Implement MongoDB integration (connection ready)

2. Image Storage

- Images stored as Data URLs (not scalable)
- **Solution:** Implement cloud storage (AWS S3, Cloudinary)

3. Authentication

- Basic admin panel without proper auth
- **Solution:** Implement JWT-based authentication

4. Mobile Optimization

- Some components need mobile responsiveness improvements
- **Solution:** Review and enhance mobile layouts

Immediate TODOs

High Priority

- ☐ Connect MongoDB database
- ☐ Implement cloud image storage
- ☐ Add proper authentication system
- ☐ Create user registration/login flow
- ☐ Add payment integration (Stripe configured)

Medium Priority

- ☐ Implement email notifications
- ☐ Add real-time chat system
- ☐ Create mobile app (React Native)
- ☐ Add video call integration
- ☐ Implement advanced matching algorithm

Low Priority

- ☐ Add multi-language support
 - ☐ Create admin analytics dashboard
 - ☐ Add automated testing
 - ☐ Implement PWA features
 - ☐ Add social media integration
-

Future Enhancements

Phase 1: Core Functionality (1-2 months)

1. Database Integration

- MongoDB Atlas connection
- User authentication system
- Real user profiles and matching

2. Payment System

- Stripe integration completion
- Subscription management
- Invoice generation

3. Communication System

- In-app messaging
- Email notifications
- WhatsApp integration

Phase 2: Advanced Features (2-3 months)

1. AI Matching Algorithm

- Machine learning integration
- Compatibility scoring
- Behavior analysis

2. Mobile Application

- React Native app
- Push notifications
- Offline capabilities

3. Video Integration

- Virtual meetings
- Video profiles
- Live streaming events

Phase 3: Scale & Optimize (3-6 months)

1. Performance Optimization

- CDN integration
- Caching strategies
- Database optimization

2. Advanced Analytics

- User behavior tracking
- Success rate analysis
- A/B testing framework

3. Enterprise Features

- White-label solutions
 - API for third parties
 - Advanced admin controls
-

Support & Maintenance

Code Maintenance

- **Framework Updates:** Regularly update Next.js and dependencies
- **Security Patches:** Monitor and apply security updates
- **Performance Monitoring:** Use tools like Vercel Analytics or Google PageSpeed

Content Management

- **Blog System:** Fully functional CMS for content updates
- **SEO Management:** Automated meta generation with manual override
- **Image Optimization:** Next.js automatic image optimization

Monitoring & Alerts

- **Error Tracking:** Implement Sentry or similar
 - **Performance Monitoring:** Vercel Analytics or Google Analytics
 - **Uptime Monitoring:** Set up alerts for downtime
-

Documentation & Resources

Developer Resources

- **Next.js Documentation:** <https://nextjs.org/docs>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **TypeScript:** <https://www.typescriptlang.org/docs>
- **React Quill:** <https://github.com/zenoamaro/react-quill>

Design Resources

- **Color Palette:** Defined in `tailwind.config.js`
- **Typography:** Google Fonts (Inter, Playfair Display)
- **Icons:** Lucide React icon library
- **Images:** Unsplash integration configured

SEO Resources

- **Schema.org:** Structured data implementation
 - **Open Graph:** Social media optimization
 - **Google Search Console:** SEO monitoring
 - **Core Web Vitals:** Performance metrics
-

Project Handover Checklist

Completed Items

- ☒ Full Next.js application setup
- ☒ Responsive design implementation
- ☒ Blog management system with rich text editor
- ☒ SEO optimization with structured data
- ☒ Admin panel with CRUD operations
- ☒ Typography and color system
- ☒ Image optimization setup
- ☒ API routes for blog management
- ☒ Production build configuration

- ✓ Deployment-ready codebase

Handover Requirements

- ✓ Complete documentation
- ✓ Code comments and explanations
- ✓ Environment configuration guide
- ✓ Deployment instructions
- ✓ Database schema documentation
- ✓ Component library documentation
- ✓ API documentation
- ✓ Future development roadmap

Next Steps for New Developer

1. Environment Setup (Day 1)

- Clone repository and install dependencies
- Configure environment variables
- Run local development server
- Test all major features

2. Code Familiarization (Week 1)

- Review component structure
- Understand data flow
- Test admin panel functionality
- Explore API endpoints

3. Priority Implementation (Month 1)

- Connect MongoDB database
- Implement user authentication
- Set up cloud image storage
- Deploy to production

Document Version: 1.0

Last Updated: January 2025

Created By: Shivam Rai

Project Status: Ready for Handover ✓

This document contains all necessary information for a smooth project handover. For any clarifications or additional information, please refer to the codebase comments and inline documentation.